

Práctica No. 4 - API de Hilos

Integrantes

Nombre: Cristian Flórez

Correo: cristian.florez@udea.edu.co

Documento: 1040183014

Video de sustentación:

Notebook de análisis:

Documentación de funciones

1. Cálculo de pi Paralelo (pip.c)

- **void *calcular_pi_parcial(void *arg)**
 - **Descripción:** Función ejecutada por cada hilo secundario. Realiza la integración numérica de la región del bucle asignada a su identificador de hilo para calcular una sección del valor de pi.
 - **Parámetros:** void *arg: Puntero a una estructura o un entero que representa el ID del hilo (usado para calcular los límites del paso/salto en el ciclo for).
 - **Retorno:** void *: Retorna un puntero al resultado parcial calculado de la suma (o acumula localmente para evitar condiciones de carrera, devolviendo el puntero a dicha dirección asignada dinámicamente).
- **int main(int argc, char *argv[])**
 - **Descripción:** Hilo principal (Master Thread). Se encarga de procesar los argumentos de la línea de comandos (número de hilos y número de pasos), inicializar las estructuras de datos, instanciar los hilos mediante pthread_create, sincronizar la finalización de los hilos con pthread_join, sumar los subtotales y calcular/imprimir el tiempo de ejecución con precisión de microsegundos.

2. Generación de Secuencia de Fibonacci (fibonacci.c)

- **void *generar_fibonacci(void *arg)**
 - **Descripción:** Función encargada de calcular de manera secuencial los primeros N números de la serie de Fibonacci e ir almacenando cada elemento calculado directamente en el arreglo compartido en memoria.
 - **Parámetros:** void *arg: Puntero que contiene el número de elementos N que solicitó el usuario.
 - **Retorno:** void *: NULL (los resultados se escriben directamente en el espacio de memoria compartida asignado por el hilo principal).
- **int main(int argc, char *argv[])**
 - **Descripción:** Hilo principal. Valida la entrada del usuario, reserva dinámicamente el arreglo en memoria donde se guardará la secuencia,

crea el hilo trabajador pasándole la dirección del arreglo y el límite, espera a que el hilo termine mediante `pthread_join` (mecanismo de sincronización) y, finalmente, recorre el arreglo compartido para imprimir la serie resultante por consola.

Problemas Presentados y Soluciones

1. **Problema: Error externally-managed-environment (PEP 668) al instalar dependencias de graficación en macOS.**
 - Solución: Se optó por la arquitectura limpia recomendada: se creó un entorno virtual aislado mediante el comando `python3 -m venv env`, lo que permitió realizar un `pip install` libre de restricciones del sistema operativo, garantizando la portabilidad del laboratorio.
2. **Problema: Desconexión de Kernels en VS Code (ModuleNotFoundError).**
 - Solución: Aunque las librerías se instalaron en la terminal, el Notebook no las detectaba. Se procedió a registrar de manera explícita el entorno virtual como un Jupyter Kernel global usando `python3 -m ipykernel install --user --name=env` y se seleccionó manualmente el Kernel activo en la esquina superior derecha del editor.
3. **Problema: Condiciones de carrera o sobreescritura de datos compartidos en pip.c.**
 - Solución: Se evitó que múltiples hilos modificaran de manera simultánea una única variable global `sum`. En su lugar, se implementó un esquema de acumulación local donde cada hilo calcula de manera independiente su sumatoria en variables locales y, al hacer el `pthread_join`, el hilo principal recopila de forma segura los retornos individuales.

Pruebas y Verificación de Funcionalidad

1. **Prueba de Consistencia Matemática (π):**
 - Se ejecutó `pis` (serial) y `pip` (paralelo) variando desde 1 hasta 8 hilos con un valor fijo de pasos (1,000,000,000). Se verificó que, independientemente del nivel de paralelismo implementado, el valor resultante de π se mantuviera idéntico con sus decimales de precisión tradicionales (3.14159265...), validando la ausencia de corrupción de datos.
2. **Prueba de Concurrencia Extrema (Fibonacci):**
 - Se probó el programa `fibonacci` con límites válidos ($N=10$, $N=40$) y límites inválidos o extremos (N menor o igual a 0). El hilo máster bloqueó la impresión de datos secuenciales de manera exacta hasta recibir la señal de finalización (`pthread_join`) del hilo hijo, garantizando que jamás se intentara leer memoria vacía o indeterminada.

Manifiesto de Transparencia

- **Uso de IA Generativa:** Se utilizó un modelo de Inteligencia Artificial generativa exclusivamente como herramienta de soporte técnico para el diagnóstico y resolución de errores de configuración del entorno de ejecución local en sistemas macOS (como la gestión del bloqueo de paquetes PEP 668 y la vinculación manual del kernel de Jupyter con VS Code). Toda la lógica de paralelización en C, el diseño de estructuras de datos mediante Pthreads y la interpretación de métricas de rendimiento fueron desarrollados de manera autónoma.

Conclusiones

1. **Impacto del Overhead de Creación de Hilos:** A niveles bajos de carga de trabajo, el tiempo requerido por el Sistema Operativo para reservar recursos, crear y destruir estructuras de hilos (`pthread_create/pthread_join`) supera el beneficio del paralelismo, provocando que la ejecución con 1 hilo paralelo sea ligeramente más lenta que la versión puramente serial.
2. **Límite de Escalabilidad Física (Ley de Amdahl):** El Speedup de un programa no escala infinitamente con el número de hilos. Una vez que el número de hilos supera la cantidad de núcleos físicos reales de la CPU (por ejemplo, en arquitecturas Apple Silicon), la curva de velocidad se aplana debido al cambio de contexto (Context Switch) y la competencia por el acceso a la memoria caché.
3. **Eficiencia Decreciente:** La eficiencia del hardware decrece a medida que añadimos más hilos de procesamiento. Mientras que con un solo núcleo la eficiencia roza el 100% (1.0), al distribuir el trabajo en 8 hilos la eficiencia disminuye significativamente, reflejando que los recursos gastan tiempo coordinándose en lugar de computar.
4. **Importancia Crítica de la Sincronización:** El uso estricto de llamadas de sincronización como `pthread_join` es indispensable en el modelo de memoria compartida. En programas como `fibonacci.c`, omitir esta espera provocaría que el hilo padre consuma e imprima un arreglo vacío o con datos parciales basura, desatando errores de carrera (Race Conditions).
5. **Aislamiento de Entornos como Buena Práctica:** El desarrollo moderno en sistemas operativos requiere un control riguroso de dependencias. Enfrentar problemas del sistema operativo base (como el entorno administrado externamente en macOS) demuestra el valor de los entornos virtuales (`venv`), asegurando que las herramientas de análisis de datos funcionen de manera portable y repetible.